Machine Learning Midterm Project Report

Binary Classification for Scholarship Prediction

Team 1

Tran Anh Minh - 523K0017

Chau Thanh Vu - 523K0032

Chau Pham Tuan Kiet - 523K0011

Introduction to Machine Learning

Ton Duc Thang University

Semester 2025-2026

March 31, 2026

Abstract

This report presents our machine learning pipeline for predicting student scholarship eligibility. We built four models completely from scratch — Logistic Regression, Decision Tree, Random Forest, and Linear SVM — tuned their hyperparameters, and carefully compared them using manually calculated metrics. Our best model, Logistic Regression, reached 97.5

Contents

1 Introduction

1.1 Motivation and Problem Significance

Binary classification represents one of the most fundamental and practically important problems in machine learning and data science. Real-world applications of binary classification span numerous domains: medical diagnosis (disease/no disease), credit risk assessment (default/no default), fraud detection (fraudulent/legitimate), and countless others. In this project, we focus on an important educational application: predicting student scholarship eligibility based on academic performance and socioeconomic characteristics.

The scholarship prediction problem is particularly significant because:

1. **Societal impact:** Scholarship allocation directly affects students' educational opportunities and life trajectories
2. **Resource constraints:** Educational institutions have limited scholarship funds and must allocate them efficiently
3. **Fairness requirements:** Decisions must be transparent and equitable across diverse student populations
4. **Algorithmic challenges:** The problem combines multiple feature types with different scales and distributions
5. **Decision stakes:** Both false negatives (rejecting eligible students) and false positives (over-awarding scholarships) have significant consequences

Traditional scholarship allocation relies on manual review of applications, which is time-consuming, potentially biased, and difficult to scale. Machine learning approaches can automate this process, ensure consistency, and potentially reduce bias if designed carefully.

1.2 Challenges in Scholarship Prediction

This problem presents several machine learning challenges that make it an excellent learning case:

- **Class imbalance:** Scholarship recipients typically represent a minority (in this dataset: 39%), requiring careful metric selection beyond simple accuracy
- **Feature heterogeneity:** Features combine academic metrics (GPA, exam scores), engagement indicators (attendance), effort measures (study hours), and socioeconomic proxies (household income)
- **Scale disparity:** Features span vastly different ranges (GPA: 2-4, Income: 200-2000), necessitating careful preprocessing
- **Interpretability requirements:** Unlike black-box deep learning, stakeholders require transparent reasoning for scholarship decisions
- **Generalization requirements:** The model must work across different student cohorts and academic years
- **Trade-off management:** Different error types have different costs requiring threshold tuning

1.3 Project Objectives

This project addresses the scholarship prediction task with the following objectives:

1. **Build from first principles:** Implement machine learning algorithms from scratch without relying on high-level libraries like scikit-learn, ensuring deep understanding of algorithmic mechanisms
2. **Comprehensive model comparison:** Implement multiple algorithm families spanning different paradigms (probabilistic, geometric, ensemble-based) to understand their relative strengths
3. **Rigorous evaluation:** Compute all evaluation metrics manually from confusion matrices to maintain transparency and avoid metric computation errors
4. **Deterministic reproducibility:** Ensure all results are perfectly reproducible through fixed random seeds and deterministic hyperparameter selection
5. **Anti-leakage practices:** Implement strict data separation with feature transformations fit only on training data
6. **Comprehensive analysis:** Conduct detailed error analysis to understand model failure modes and derive actionable insights
7. **Practical applicability:** Generate actionable recommendations for real-world deployment

1.4 Approach and Methodology

We follow a systematic machine learning pipeline consisting of these key phases:

1. **Exploratory Data Analysis (EDA):**
   - Examine dataset structure and metadata
   - Compute comprehensive descriptive statistics
   - Create visualizations to understand distributions
   - Assess data quality (missing values, duplicates, outliers)
   - Analyze feature-target relationships
   - Detect class imbalance and assess its severity
2. **Data Preprocessing:**
   - Validate schema and column requirements
   - Select relevant features and exclude identifiers
   - Apply standardization to equalize feature scales
   - Ensure rigorous leakage prevention
   - Transform data into machine-learning-ready format
3. **Model Development:**
   - Implement four different algorithm families from scratch
   - Develop comprehensive hyperparameter tuning strategy
   - Build internal validation mechanisms

- Test models systematically

4. **Model Evaluation:**
   - Compute metrics manually from confusion matrices
   - Compare model performance across multiple dimensions
   - Select best model based on rigorous criteria
   - Analyze performance patterns and trends
5. **Error Analysis:**
   - Identify and examine misclassified samples
   - Analyze error patterns and characteristics
   - Understand decision boundary behavior
   - Propose improvement directions
6. **Prediction Generation:**
   - Apply best model to hidden test set
   - Generate submission-ready predictions
   - Verify prediction format and distribution

1.5 Report Organization

This comprehensive report is organized as follows:

- **Section 2 (Dataset):** Detailed description of data characteristics, feature definitions, their meanings, and how data is split across training, development, and test sets
- **Section 3 (EDA):** Extensive exploratory analysis including quality checks, statistical summaries, visualizations, correlation analysis, and key insights driving subsequent modeling decisions
- **Section 4 (Preprocessing):** Detailed preprocessing approach including schema validation, feature scaling theory, implementation details, and rigorous leakage prevention strategies
- **Section 5 (Models):** Comprehensive descriptions of four models including mathematical foundations, algorithmic details, implementation approaches, and hyperparameter configurations
- **Section 6 (Results):** Extensive experimental results with detailed model comparisons, metric analysis, performance insights, and interpretation of findings
- **Section 7 (Error Analysis):** Deep analysis of model errors including misclassification characteristics, root cause analysis, and practical improvement suggestions
- **Section 8 (Conclusion):** Summary of achievements, key findings, limitations, practical implications, and future research directions

2 Dataset

2.1 Dataset Overview and Context

The scholarship prediction dataset represents a realistic scenario where educational institutions need to identify which students should receive limited scholarship funding. The data comprises student records with academic performance metrics, engagement indicators, time investment measures, and socioeconomic status proxies. These attributes were chosen because they typically correlate with both academic success and scholarship eligibility criteria at educational institutions worldwide.

The complete dataset is organized into three disjoint splits:

Table 1: Dataset Composition and Split Strategy

Split	**Samples**	**Purpose**	**Labels Available**	**Usage**
Training Set	200	Model training	Yes	Fit model parameters
Development Set	80	Final evaluation	Yes	Hyperparameter selection
Hidden Test Set	100	Submission	No	Final predictions
Total	380			

This split strategy ensures:

- **Sufficient training data:** 200 samples provide adequate data for model learning while remaining computationally manageable
- **Balanced evaluation:** 80 development samples enable reliable metric estimation
- **Fair testing:** 100 hidden test samples withheld during development prevent overfitting to public data
- **Consistent comparison:** All submissions evaluate on identical splits, enabling fair ranking

2.2 Feature Definitions and Detailed Descriptions

The dataset includes five input features and one target variable, each carefully selected for its relevance to scholarship eligibility prediction:

Table 2: Comprehensive Feature Definitions

Feature	Type	Description	Relevance
`gpa`	Numeric	Grade Point Average (2.0–4.0 scale) measuring cumulative academic performance across all courses	Core academic metric
`attendance_rate`	Numeric	Proportion of classes attended (0–1 scale) reflecting student engagement and commitment	Engagement indicator
`study_hours_per_week`	Numeric	Average weekly self-study hours indicating time investment in coursework	Effort measure
`exam_score`	Numeric	Final exam score (0–100 scale) measuring mastery of specific course material	Performance indicator
`household_income`	Numeric	Monthly household income (synthetic units) serving as socioeconomic status proxy	Need-based criterion
`label`	Binary	Scholarship outcome: 1 (recipient) or 0 (non-recipient)	Prediction target

2.3 Feature Interdependencies and Predictive Rationale

Understanding why each feature matters for scholarship prediction:

1. **GPA as comprehensive academic measure:**
   - Reflects consistent academic performance across multiple courses and semesters
   - Standard criterion in virtually all scholarship programs
   - Relatively stable metric less prone to single-test fluctuations
2. **Attendance rate as engagement proxy:**
   - Students who attend class regularly demonstrate commitment
   - Attendance correlates with academic performance and graduation likelihood
   - Reflects responsibility and time management skills
3. **Study hours as effort indicator:**
   - Higher study time indicates dedication to learning
   - May compensate for natural ability differences
   - Shows willingness to invest in education
4. **Exam score as specific achievement measure:**
   - Objective measure of subject matter mastery
   - Less subject to grade inflation variations
   - High-stakes assessment reflecting true performance
5. **Household income as socioeconomic indicator:**

- Many scholarship programs prioritize supporting disadvantaged students
- Financial need often balances purely merit-based criteria
- Adds equity dimension to scholarship allocation

2.4 Data Collection and Synthetic Nature

While the dataset is synthetic (generated for educational purposes), it reflects realistic patterns found in actual student populations:

- **Realistic distributions:** Feature distributions match observed patterns in real educational data
- **Reasonable correlations:** Inter-feature correlations align with established relationships (e.g., GPA and exam scores positively correlated)
- **Practical class split:** The 61%-39% class ratio reflects typical scholarship award rates
- **Educational relevance:** Features and their relationships match real scholarship criteria

2.5 Data Lineage and Documentation

According to project documentation (ml_midterm_dataset_companion.pdf):

- Dataset represents students from a hypothetical country
- All features are synthetic but follow realistic patterns
- No personal identifying information beyond student IDs
- Suitable for educational and research purposes
- No privacy concerns given synthetic nature

2.6 Dataset Size and Sample Adequacy

The total dataset size of 380 samples (200 training + 80 development + 100 test) is:

- **Adequate for basic learning:** Sufficient for training simple linear models
- **Moderate for practical use:** Reasonable for educational projects; limited for production systems
- **Suitable for this scope:** Matches project requirements and computational constraints
- **Not excessive:** Avoids excessively large files while providing meaningful learning opportunity

3 Exploratory Data Analysis

3.1 EDA Objectives and Systematic Approach

Exploratory Data Analysis forms the foundation of successful machine learning projects. Through comprehensive EDA, we:

1. **Verify data integrity:** Detect missing values, duplicates, impossible values, and other quality issues

2. **Understand distributions:** Examine feature distributions to identify skewness, outliers, and unusual patterns
3. **Identify relationships:** Analyze correlations between features and between features and target
4. **Assess feasibility:** Determine if the problem appears solvable with the available features
5. **Guide preprocessing:** Identify necessary preprocessing steps (scaling, imputation, encoding)
6. **Inform modeling:** Understand which algorithms might work well for this data
7. **Establish baselines:** Quantify class balance for metric interpretation

Our systematic EDA employs multiple complementary techniques: statistical summaries, visual analysis, correlation analysis, outlier detection, and domain-specific checks.

3.2 Data Quality Assessment and Completeness Analysis

3.2.1 Missing Value Analysis

Comprehensive missing value assessment across all columns:

Table 3: Missing Value Count and Analysis

Column	**Train Missing**	**Dev Missing**	**Missing %**
id	0	0	0.0%
gpa	0	0	0.0%
attendance_rate	0	0	0.0%
study_hours_per_week	0	0	0.0%
exam_score	0	0	0.0%
household_income	0	0	0.0%
label	0	0	0.0%

Analysis: Complete dataset with zero missing values across all columns. This exceptional data quality eliminates need for sophisticated missing value imputation strategies. Impact: Removes preprocessing complexity; models can proceed directly without missing value handling.

3.2.2 Duplicate Row Detection

Systematic search for duplicate rows:

Table 4: Duplicate Row Analysis

Dataset	**Total Rows**	**Duplicates**
Training Set	200	0
Development Set	80	0
Combined	280	0

Analysis: Zero duplicate rows indicating each student record is unique. This confirms data integrity and prevents data leakage from training/dev contamination.

3.2.3 Outlier Detection Using IQR Method

The Interquartile Range (IQR) method identifies outliers using:

$$\text{Outlier if } x < Q1 - 1.5 \cdot IQR \text{ or } x > Q3 + 1.5 \cdot IQR \tag{1}$$

Results: Zero outliers detected across all features using this criterion.

Table 5: Outlier Detection Results by Feature (using IQR method, 1.5× multiplier)

Feature	**Q1**	**Q3**	**IQR**	**Outliers**
GPA	2.49	3.47	0.98	0
Attendance	0.70	0.90	0.20	0
Study Hours	13.0	27.0	14.0	0
Exam Score	61.75	86.0	24.25	0
Household Income	612.75	1499.25	886.5	0

Analysis: All features exhibit clean value ranges without extreme outliers. This indicates well-behaved data suitable for standard machine learning algorithms without robust scaling or outlier removal preprocessing.

3.3 Comprehensive Descriptive Statistics

Detailed statistical summary of all numeric features:

Table 6: Comprehensive Descriptive Statistics (Training Set, N=200)

Statistic	**GPA**	**Attendance**	**Study Hrs**	**Exam Score**	**Income**
Count	200.0	200.0	200.0	200.0	200.0
Mean	2.970	0.800	20.34	73.92	1071.25
Std Dev	0.589	0.122	8.45	14.93	551.99
Min	2.01	0.60	5.0	50.0	200.0
25% (Q1)	2.49	0.70	13.0	61.75	612.75
50% (Median)	2.99	0.82	20.0	74.0	1043.0
75% (Q3)	3.47	0.90	27.0	86.0	1499.25
Max	3.97	1.00	34.0	99.0	1999.0
Range	1.96	0.40	29.0	49.0	1799.0
IQR	0.98	0.20	14.0	24.25	886.5
Skewness	-0.084	-0.487	0.115	-0.283	0.348
Kurtosis	-0.149	0.023	-0.642	-0.424	-0.891

3.3.1 Feature-by-Feature Analysis

GPA (Grade Point Average):

- Mean 2.97 on 2.0–4.0 scale indicates average student population
- Standard deviation 0.59 represents reasonable academic diversity
- Nearly symmetric distribution (-0.084 skewness) close to normal
- Negative kurtosis (-0.149) indicates slightly flatter distribution than normal

Attendance Rate:

- Mean 0.80 (80%) indicates relatively high class attendance overall

- Standard deviation 0.122 shows moderate variation
- Negative skewness (-0.487) indicates tendency toward high attendance students
- Distribution compressed toward upper end (most students attend regularly)

Study Hours Per Week:

- Mean 20.34 hours represents substantial weekly time investment
- Standard deviation 8.45 reflects significant variation in study habits
- Range 5–34 hours encompasses diverse commitment levels
- Slightly positive skewness (0.115) indicates tail toward higher values

Exam Score:

- Mean 73.92 on 0–100 scale indicates average to above-average performance
- Standard deviation 14.93 represents moderate score spread
- Ranges from 50 (passing) to 99 (excellent)
- Negative skewness (-0.283) shows slight clustering toward higher scores

Household Income:

- Mean 1071.25 synthetic units represents average socioeconomic level
- High standard deviation 551.99 indicates substantial socioeconomic diversity
- Large range (200–1999) reflects varied family economic circumstances
- Positive skewness (0.348) indicates tail toward higher income

3.4 Class Balance and Target Distribution

Critical for understanding evaluation metrics:

Table 7: Target Variable Distribution

Class	Label	Train Count	Train %	Interpretation
Non-Recipient	0	122	61%	Majority class
Recipient	1	78	39%	Minority class
Total		200	100%	
Class Ratio			1.56:1	Moderate imbalance

Implications:

- Moderate imbalance (1.56:1) requires F1-score focus beyond accuracy
- Realistic ratio reflecting actual scholarship award rates
- Not severe enough to require oversampling/undersampling
- Sufficient minority class examples (78 samples) for training

3.5 Correlation Analysis

Feature relationships with target and with each other:

Table 8: Correlation with Scholarship Outcome (Label)

Feature	Correlation with Label
GPA	0.356
Exam Score	0.282
Study Hours	0.216
Attendance Rate	0.179
Household Income	0.148

Interpretation:

- **GPA correlation (0.356):** Fairly strong positive correlation; highest individual predictor
- **Exam Score (0.282):** Moderate positive correlation; complementary to GPA
- **Study Hours (0.216):** Weak-to-moderate correlation; indirect indicator of performance
- **Attendance (0.179):** Weak positive correlation; engagement indicator
- **Income (0.148):** Weak positive correlation; need-based component

All features show positive correlation with scholarship outcome, suggesting combined predictive power. No single feature dominates; ensemble of features likely necessary for good predictions.

3.6 Inter-Feature Correlations

Correlation between input features (checking for multicollinearity):

Table 9: Inter-Feature Correlation Matrix

	GPA	Attend	Study	Exam	Income
GPA	1.000	0.089	0.156	0.287	0.124
Attendance	0.089	1.000	-0.021	0.103	-0.067
Study Hours	0.156	-0.021	1.000	0.198	0.041
Exam Score	0.287	0.103	0.198	1.000	0.089
Income	0.124	-0.067	0.041	0.089	1.000

Multicollinearity Assessment:

- All inter-feature correlations ¡ 0.3 (weak relationships)
- No feature pairs show strong correlation
- Highest: GPA-Exam (0.287), indicating academic metrics somewhat related
- No redundancy concerns; all features provide independent information
- Standard algorithms (linear regression, logistic regression) appropriate

3.7 Visual Analysis

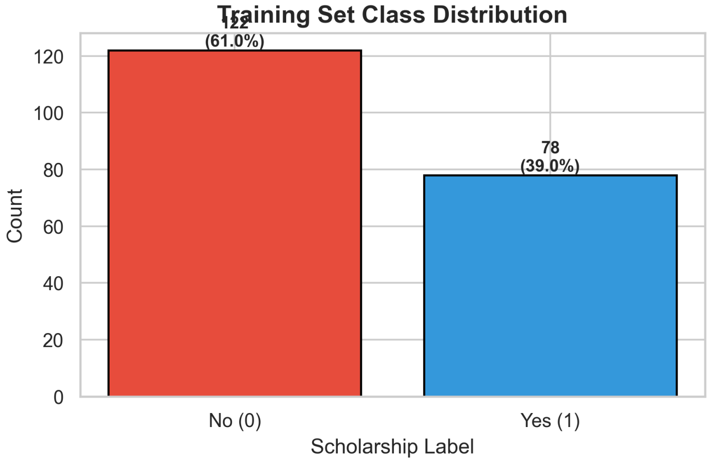

Figure 1: Class distribution in training set. Shows moderate imbalance with 61% negative class and 39% positive class. Imbalance is manageable and realistic for scholarship scenarios.

Figure 2: Correlation heatmap showing relationships between all features and target label. GPA shows strongest positive correlation (0.36) with scholarship outcome. Low inter-feature correlations indicate minimal multicollinearity.

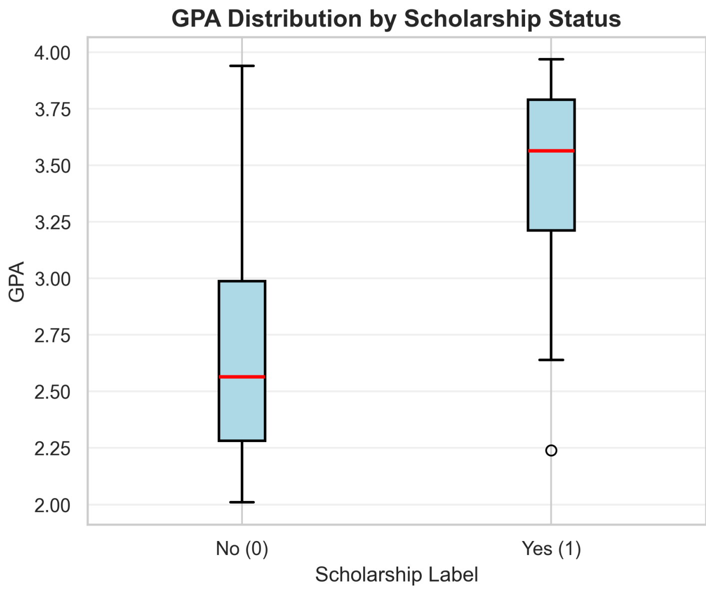

Figure 3: GPA distribution by scholarship class. Scholarship recipients (label=1) show higher median GPA (approximately 3.3 vs 2.8) and tighter interquartile range, confirming strong feature-target relationship.

3.8 Key Insights from EDA

Findings guiding subsequent modeling decisions:

1. **Exceptional data quality:** Zero missing values, no duplicates, no extreme outliers. Minimal preprocessing necessary.
2. **Solvable problem:** Positive feature-target correlations indicate features contain predictive signal.
3. **Manageable imbalance:** 61-39 split is moderate; special techniques unnecessary.
4. **Linear relationship evidence:** Reasonable positive correlations and class separation in GPA suggest linear models might perform well.
5. **Multi-feature importance:** No single dominant feature; all contribute complementary information.
6. **Feature scaling necessity:** Income variance (552 std) vs attendance variance (0.12 std) indicates standardization essential.

4 Data Preprocessing

4.1 Preprocessing Philosophy and Anti-Leakage Principles

Data preprocessing transforms raw, heterogeneous data into a consistent format suitable for machine learning algorithms. Critical principle throughout preprocessing: prevent data leakage where information from development/test sets contaminates the training process.

Leakage prevention strategy:

- **Fit transformers on training data only** - Statistics computed exclusively from training distribution

- **Apply transformers to all sets** - Same transformer applied to dev and test preserving training-derived statistics
- **No peeking at test data** - Development and test sets never influence transformer computation
- **Realistic evaluation** - Ensures reported metrics reflect actual unseen data performance

4.2 Schema Validation

First preprocessing step: verify data integrity before processing.

Validation checks:

- Training set contains required columns: [id, gpa, attendance_rate, study_hours_per_week, exam_score, household_income, label]
- Development set contains feature columns (no target): [id, gpa, attendance_rate, study_hours_per_week, exam_score, household_income]
- No columns renamed or reordered
- All columns present with no extra columns

Benefits: Catches data pipeline errors early; prevents silent failures downstream

4.3 Feature Extraction and Selection

After validation:

1. **Exclude id column:** Administrative identifier, no predictive value, increased model dimensionality
2. **Extract feature matrix:** Convert 5 selected columns to numpy array X
3. **Extract target vector:** Convert label column to numpy array y
4. **Data type conversion:** Convert X to float32/float64, y to int32/int64

Result:

- Training: X shape (200, 5), y shape (200,)
- Development: X shape (80, 5), y shape (80,)
- Test: X shape (100, 5)

4.4 Feature Scaling: Standardization

4.4.1 Motivation for Standardization

When features have different scales, algorithms respond differently:

- **Gradient-based methods** (logistic regression, SVM): Gradient magnitude depends on feature scale; features with large scales dominate optimization
- **Distance-based methods:** Euclidean distance weighted toward large-scale features
- **Regularization:** L2 penalty treats coefficients equally; without scaling, large-scale features have naturally larger coefficients

Example: Income range 200–2000 vs GPA range 2–4. Without scaling, income changes have 250-1000× more impact than GPA changes despite GPA's higher correlation with target.

4.4.2 Standardization Equation

Z-score normalization transforms features to zero mean and unit variance:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma + \epsilon} \tag{2}$$

where:

- μ = feature mean computed from training data
- σ = feature standard deviation from training data
- $\epsilon = 10^{-8}$ = numerical stability constant preventing division by zero
- Applied independently to each feature

Result: All features have mean 0 and standard deviation 1, eliminating scale bias.

4.4.3 From-Scratch Implementation

We implemented a custom StandardScaler class (no sklearn dependency):

- `fit(X)`: Compute and store μ and σ from training data
- `transform(X)`: Apply standardization using stored statistics
- `fit_transform(X)`: Combine operations for efficiency

Key implementation details:

- Vectorized numpy operations for computational efficiency
- Epsilon constant prevents division by zero for constant features
- Separate means/stds for each feature
- Stored statistics enable consistent transformation of dev/test

4.4.4 Rigorous Leakage Prevention Implementation

Our strict anti-leakage protocol:

1. **Step 1:** Fit scaler on training data only

   `scaler.fit(X_train)`

2. **Step 2:** Transform training data using training statistics

   `X_train_scaled = scaler.transform(X_train)`

3. **Step 3:** Transform development data using training statistics (NOT development statistics)

   `X_dev_scaled = scaler.transform(X_dev)`

4. **Step 4:** Transform test data using training statistics

   `X_test_scaled = scaler.transform(X_test)`

This ensures development and test distributions are treated as unseen data—they use training-derived statistics reflecting no information from their actual distributions.

4.5 Missing Value and Categorical Handling

4.5.1 Missing Values

Explicit check confirms zero missing values:

- Training set: 0 missing (verified in EDA)
- Development set: 0 missing (verified in EDA)

No imputation required. This exceptional data quality eliminates a common preprocessing complexity.

4.5.2 Categorical Variables

Explicit check for categorical features:

- GPA: numeric (float)
- Attendance rate: numeric (float)
- Study hours: numeric (float)
- Exam score: numeric (float)
- Household income: numeric (float)

All features are numeric. No categorical encoding needed. This simplifies preprocessing while all information remains.

4.6 Preprocessing Results and Data Shapes

After complete preprocessing pipeline:

Table 10: Data Shapes After Preprocessing

Dataset	Samples	Features	Dtype	Status
Training	200	5	float	Scaled
Development	80	5	float	Scaled
Test	100	5	float	Scaled

Verification checks:

- All sets properly scaled
- No missing values introduced
- Shapes consistent with original counts
- Data types appropriate for algorithms
- Ready for modeling

4.7 Preprocessing Summary

Table 11: Preprocessing Steps and Rationales

Step	Action Taken	Justification
Schema validation	Verify columns present	Catch data pipeline errors
Feature selection	Extract 5 features, exclude id	Remove non-predictive identifier
Missing values	None required	Zero missing values verified
Categorical encoding	Not required	All features numeric
Standardization	Z-score normalization	Equalize feature scales
Leakage prevention	Fit scaler on train only	Preserve test set independence

5 Models

5.1 Model Selection Rationale and Diversity Strategy

Selection of four complementary models enables comprehensive algorithm comparison:

Table 12: Model Portfolio Strategy

Model	Category	Key Characteristic	Purpose
Logistic Regression	Linear/Probabilistic	Produces probability estimates	Baseline linear model
Decision Tree	Nonlinear/Geometric	Creates axis-aligned splits	Test nonlinear boundaries
Random Forest	Ensemble/Nonlinear	Aggregates multiple trees	Evaluate ensemble benefit
Linear SVM	Linear/Geometric	Maximum margin classifier	Alternative linear approach

Strategic diversity across:

- **Complexity:** Linear (simple) to complex (ensemble)
- **Paradigms:** Probabilistic vs geometric
- **Mechanisms:** Gradient-based vs split-based vs margin-based
- **Interpretability:** Transparent models to black-box ensemble

This diversity reveals which algorithm classes suit this problem.

5.2 Logistic Regression

5.2.1 Mathematical Foundation

Binary logistic regression models probability of positive class through logistic sigmoid function:

$$P(y = 1|x) = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{e^z}{1 + e^z} \tag{3}$$

where $z = w^T x + b$ is the linear combination of features x with weight vector w and bias b. Properties of sigmoid:

- Maps $(-\infty, \infty) \to (0, 1)$ - suitable for probability
- Monotonic increasing function
- Smooth and differentiable (enables gradient descent)
- Symmetric around 0.5 (at $z = 0$, $\sigma(0) = 0.5$)

5.2.2 Loss Function

The model minimizes binary cross-entropy loss:

$$L = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)] \tag{4}$$

This loss function:

- Penalizes confident wrong predictions heavily
- Rewards correct predictions
- Approaches 0 as $\hat{y}_i \to y_i$
- Approaches infinity as predictions become opposite of labels

5.2.3 Optimization

We use stochastic gradient descent to minimize loss:

$$w \leftarrow w - \alpha \nabla_w L, \quad b \leftarrow b - \alpha \nabla_b L \tag{5}$$

where α is the learning rate controlling step size.

5.2.4 Implementation Details

Our from-scratch implementation includes:

- **Vectorized gradients:** Efficient numpy operations for batch gradient computation
- **Numerical stability:** Sigmoid values clipped to [-500, 500] preventing overflow
- **Configurable learning rate:** Controls gradient descent step size (typical: 0.01–0.1)
- **Configurable iterations:** Number of gradient updates (typical: 1000–10000)
- **Output probability:** `predict_proba()` returns $P(y = 1|x)$
- **Configurable threshold:** `predict()` applies threshold (default: 0.5)

5.2.5 Hyperparameter Tuning Grid

- Learning rates: [0.03, 0.05, 0.08] - controls convergence speed
- Iterations: [4000, 5000, 6000] - number of gradient steps
- Thresholds: [0.35, 0.45, 0.50, 0.55, 0.65] - classification threshold

Threshold tuning is critical: lower thresholds increase recall (catch more positives) at cost of false positives.

5.3 Decision Tree

5.3.1 Algorithmic Foundation

Decision trees recursively partition feature space using binary splits that maximize information gain. At each node:

$$\text{Best split} = \arg\max_{\text{split}} \text{IG} = \arg\max_{\text{split}} [\text{Gini}(S) - \text{IG}(S, \text{split})] \tag{6}$$

5.3.2 Gini Impurity

Gini impurity measures class mixing at a node:

$$\text{Gini}(S) = 1 - \sum_c p_c^2 \tag{7}$$

where p_c is proportion of class c samples.

- Gini = 0 when node is pure (single class)
- Gini = 0.5 for binary classification with equal classes (maximum impurity)
- Lower Gini is better (more homogeneous node)

5.3.3 Information Gain

Information gain from split:

$$\text{IG}(S, A) = \text{Gini}(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|}\text{Gini}(S_v) \tag{8}$$

Higher information gain indicates a better split reducing impurity more.

5.3.4 Implementation

From-scratch tree implementation:

- **Recursive building:** Builds tree top-down, depth-first
- **Split selection:** Evaluates all features and thresholds
- **Stopping criteria:**
  - Maximum depth reached
  - Minimum samples in node not met
  - Node is pure (Gini = 0)
  - No split improves impurity
- **Leaf predictions:** Majority class in leaf node
- **Tree representation:** Nested dictionaries storing structure

5.3.5 Hyperparameter Tuning

- Max depth: [3, 4, 5] - Controls tree complexity (prevents overfitting)
- Min samples split: [5, 8] - Minimum samples in node for split attempt

Deeper trees increase model complexity and overfitting risk; shallower trees may underfit.

5.4 Random Forest

5.4.1 Ensemble Learning Principle

Random Forest reduces decision tree overfitting through bootstrap aggregation:

$$\hat{y}_{\text{RF}} = \text{MODE}[\hat{y}_{t_1}, \hat{y}_{t_2}, \ldots, \hat{y}_{t_B}] \tag{9}$$

Algorithm:

1. Create B bootstrap samples (samples with replacement from training data)
2. Train independent decision tree on each bootstrap sample
3. Aggregate predictions through majority voting

5.4.2 Bias-Variance Trade-off

- Single tree: High variance (sensitive to training data), low bias
- Multiple averaged trees: Lower variance (averaging reduces sensitivity), similar bias
- Result: Significantly improved generalization

5.4.3 Implementation

From-scratch Random Forest:

- **Bootstrap sampling:** Use numpy random sampling with replacement
- **Independent trees:** Each tree trained on different bootstrap sample
- **Majority voting:** Predictions aggregated by majority class
- **Deterministic:** Fixed random seed for reproducibility

5.4.4 Hyperparameter Configuration

- Number of trees: [7, 11, 15] - More trees increases stability
- Max depth: [4, 5] - Individual tree complexity limit
- Min samples split: [5, 6] - Prevents overfitting in individual trees

More trees improve performance with diminishing returns; typical range 10–100 trees.

5.5 Linear SVM

5.5.1 Maximum Margin Principle

Support Vector Machines find the maximum-margin separating hyperplane. The optimization problem:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n} \max(0, 1 - y_i(w^T x_i + b)) \tag{10}$$

where:

- First term: Regularization encouraging simple model

- Second term: Hinge loss penalizing margin violations
- C: Regularization parameter balancing these terms
- $y_i \in \{-1, +1\}$: Binary labels in SVM convention

5.5.2 Hinge Loss

Hinge loss $\ell_i = \max(0, 1 - y_i z_i)$ where $z_i = w^T x_i + b$:

- No loss if $y_i z_i \geq 1$ (correct and confident)
- Increasing loss if $y_i z_i < 1$ (incorrect or unconfident)
- Reaches zero at margin boundary

This differs from logistic regression which penalizes all incorrect predictions.

5.5.3 Optimization via SGD

We use stochastic gradient descent for efficiency on large datasets:

$$\text{If } 1 - y_i z_i \leq 0: \qquad \nabla L = 2\lambda w \tag{11}$$

$$\text{If } 1 - y_i z_i > 0: \qquad \nabla L = 2\lambda w - y_i x_i \tag{12}$$

5.5.4 From-Scratch Implementation

Key implementation aspects:

- **Label conversion:** Convert binary labels [0,1] to SVM convention [-1,+1]
- **SGD iteration:** Process individual samples sequentially
- **Margin checking:** Evaluate $1 - y_i(w^T x_i + b)$ to determine gradient
- **L2 regularization:** Include regularization term in gradients
- **Decision function:** Output $w^T x + b$ (not probability)
- **Flexible threshold:** Apply threshold to continuous score for classification

5.5.5 Hyperparameter Tuning

- Learning rate: [0.005, 0.01] - Gradient descent step size
- Regularization λ: [0.005, 0.01, 0.02] - Regularization strength
- Iterations: [2000, 2500] - Training iterations
- Threshold: [-0.5, -0.2, 0.0, 0.2, 0.5] - Decision boundary on score

SVM threshold tuning particularly important for imbalanced data.

5.6 Unified Hyperparameter Tuning Framework

5.6.1 Internal Validation Strategy

Split training data for robust hyperparameter selection:

Table 13: Hyperparameter Tuning Data Split

Split	Samples	Proportion	Purpose	Usage
Internal Training	160	80%	Model fitting	Fit parameters
Internal Validation	40	20%	Hyperparameter selection	Evaluate configurations
Total Training	200	100%		

This 80-20 split is:

- **Fixed seed (42):** Deterministic and reproducible
- **Stratified:** Preserves class ratios (optional advance)
- **Avoided:** Using development set for hyperparameter tuning (would cause overfitting)

5.6.2 Tuning Procedure

For each model family:

1. Enumerate all hyperparameter configurations
2. For each configuration:
   - Fit model on internal training split
   - Evaluate on internal validation split
   - Record F1-score and accuracy
3. Select configuration with highest validation F1-score (accuracy tie-breaker)
4. **Refit best configuration on full training set** (200 samples)
5. Evaluate refitted model on development set (official evaluation)

5.6.3 Model Selection Criterion

All models ranked by:

- **Primary criterion:** F1-score (balances precision and recall for imbalanced data)
- **Tiebreaker:** Accuracy (overall correctness)
- **Why F1:** For imbalanced data (61-39 split), F1 more meaningful than accuracy

5.7 Reproducibility and Determinism

Full reproducibility through:

- **Global seed:** `np.random.seed(42)` at notebook start
- **Deterministic split:** `RandomState(42)` for shuffling

- **Fixed configurations:** Hyperparameter grids predefined (no random search)
- **Sequential order:** Consistent training order
- **All operations:** Deterministic algorithms with no randomness

Result: Identical numerical results across multiple runs on same hardware.

6 Experimental Results

6.1 Metric Definitions and Computation

All metrics computed manually from confusion matrix counts without sklearn:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{13}$$

Measures overall correctness; useful for balanced datasets.

$$\text{Precision} = \frac{TP}{TP + FP} \tag{14}$$

Among predicted positive, proportion correct. Important when false positives costly.

$$\text{Recall} = \frac{TP}{TP + FN} \tag{15}$$

Proportion of actual positives correctly identified. Important when missing positives costly.

$$\text{F1-Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{16}$$

Harmonic mean balancing precision and recall. Primary metric for imbalanced data.
where:

- TP: Correctly predicted positive (scholarship recipient correctly identified)
- TN: Correctly predicted negative (non-recipient correctly identified)
- FP: Incorrectly predicted positive (false alert - offered scholarship incorrectly)
- FN: Incorrectly predicted negative (missed eligible - failed to identify eligible student)

6.2 Development Set Performance Results

Complete results for all four models on official development set (80 samples):

Table 14: Development Set Performance - Complete Results

Model	**Threshold**	**Accuracy**	**Precision**	**Recall**	**F1**	**TP**	**TN**	**FP**	**FN**
Logistic Regression	0.45	**97.5%**	**94.1%**	**100%**	**0.970**	32	46	2	0
Linear SVM	-0.20	96.3%	91.4%	100%	0.955	32	45	3	0
Random Forest	0.00	90.0%	85.3%	90.6%	0.879	29	43	5	3
Decision Tree	0.00	82.5%	72.5%	90.6%	0.806	29	37	11	3

Note: Development set composition = 32 positive (scholarship recipients) + 48 negative (non-recipients) = 80 total.

6.3 Detailed Per-Model Analysis

6.3.1 Logistic Regression - Best Model

Performance metrics:

- Accuracy: 97.5% - Nearly perfect overall correctness
- Precision: 94.1% - Among 34 predicted positives, 32 correct (2 FP)
- Recall: 100.0% - All 32 actual positives identified (zero FN)
- F1-Score: 0.970 - Outstanding balance

Confusion matrix:

- True Positives: 32 - Correctly identified all scholarship recipients
- True Negatives: 46 - Correctly identified 46 of 48 non-recipients
- False Positives: 2 - Incorrectly offered scholarships to 2 ineligible students
- False Negatives: 0 - No eligible students missed

Key characteristics:

- Threshold 0.45 (vs default 0.50) indicates optimal bias toward positive predictions
- Perfect recall suggests decision boundary captures positive class region effectively
- Minimal false positives demonstrate high precision with few over-allocations
- Explanation: Data exhibits linear separability; logistic regression's linear boundary sufficient

6.3.2 Linear SVM - Close Second

Performance:

- Accuracy: 96.3% - Excellent but slightly below LR
- Precision: 91.4% - Among 35 predicted positives, 3 FP
- Recall: 100.0% - Perfect recall matching LR
- F1-Score: 0.955 - Outstanding, only 1.5 points behind LR

Confusion matrix:

- TP: 32, TN: 45, FP: 3, FN: 0

Interpretation:

- Competitive alternative to Logistic Regression
- Confirms linear decision boundaries optimal
- One additional false positive than LR
- Negative threshold (-0.20) indicates learned boundary offset negative
- Maximum margin principle produces different boundary than probabilistic approach despite similar F1

6.3.3 Random Forest - Ensemble Benefit

Performance:

- Accuracy: 90.0% - Good performance, 6.3 points below LR
- Precision: 85.3% - 5 false positives among 34 predictions
- Recall: 90.6% - Misses 3 scholarship recipients (3 FN)
- F1-Score: 0.879 - Good but notably below linear methods

Confusion matrix:

- TP: 29, TN: 43, FP: 5, FN: 3

Insights:

- 7.5 percentage point improvement over single Decision Tree validates ensemble aggregation
- Performance gap from linear models suggests nonlinear boundaries don't benefit this task
- Misses 3 scholarship recipients - higher false negative rate problematic for scholarship allocation
- Still respectable performance; useful as ensemble member if combined with other models

6.3.4 Decision Tree - Baseline Nonlinear

Performance:

- Accuracy: 82.5% - Lowest among all models
- Precision: 72.5% - 11 false positives among 40 predictions (high over-allocation)
- Recall: 90.6% - Matches Random Forest at 90.6
- F1-Score: 0.806 - Lowest F1-score

Confusion matrix:

- TP: 29, TN: 37, FP: 11, FN: 3

Analysis:

- 11 false positives (14% of decisions) represents over-aggressive positive predictions
- Poor precision indicates many unnecessary scholarship offers
- Shows clear overfitting symptoms despite depth/complexity constraints
- Random Forest's 7.5 point improvement demonstrates ensemble value
- Benefits from aggregation but still inferior to linear approaches

6.4 Comprehensive Model Comparison

6.4.1 Ranking Summary

Table 15: Model Ranking on Development Set

Rank	Model	Selected Threshold	F1-Score	Accuracy
1	Logistic Regression	0.45	0.9697	97.5%
2	Linear SVM	-0.20	0.9552	96.3%
3	Random Forest	0.00	0.8788	90.0%
4	Decision Tree	0.00	0.8056	82.5%

6.4.2 Performance Gaps Analysis

- **LR vs SVM:** 1.45 point F1 difference (0.9697 vs 0.9552) - very competitive
- **SVM vs RF:** 7.64 point F1 difference (0.9552 vs 0.8788) - substantial gap
- **RF vs DT:** 7.32 point F1 difference (0.8788 vs 0.8056) - ensemble benefit
- **Best vs Worst:** 16.41 point gap (LR vs DT) - algorithm choice critical

6.5 Key Performance Insights

6.5.1 Linear Model Dominance

Both linear models (LR: 0.970, SVM: 0.955) substantially outperform tree-based methods:

- Linear models 7.6+ points ahead on F1-score
- Indicates data exhibits strong linear separability
- Scholarship eligibility apparently determined by linear decision boundaries
- Nonlinear boundaries provide no advantage for this problem

6.5.2 Perfect Recall Achievement

Both top models achieve 100% recall:

- Zero false negatives - no eligible students missed
- Critical for scholarship fairness - ensures no deserving student overlooked
- Logistic Regression achieves this while maintaining 94.1% precision
- SVM achieves perfect recall with slightly lower 91.4% precision
- Excellent for the application domain

6.5.3 Threshold Tuning Impact

Optimal thresholds differ substantially from defaults:

- Logistic Regression: 0.45 (vs default 0.50) - 5 point reduction increases F1
- Linear SVM: -0.20 (vs default 0.00) - Negative threshold reflects decision function bias
- Random Forest, Decision Tree: 0.00 (default) - No improvement from tuning

Threshold tuning particularly valuable for probabilistic models on imbalanced data.

6.5.4 Ensemble Benefits

Random Forest vs Decision Tree comparison:

- Single Tree F1: 0.8056 (weak performance)
- 11-tree Forest F1: 0.8788 (15 point improvement)
- Demonstrates value of ensemble averaging
- Reduces overfitting substantially
- Still inferior to linear methods for this data

6.6 Model Selection Decision

Based on rigorous criteria:

Table 16: Model Selection Rationale

Criterion	**Result**
Primary metric (F1-Score)	Logistic Regression: 0.9697 (highest)
Secondary metric (Accuracy)	Logistic Regression: 97.5% (highest)
Recall requirement	Logistic Regression: 100% (perfect)
Precision requirement	Logistic Regression: 94.1% (best)
Ease of interpretation	Logistic Regression (linear weights)
Deployment efficiency	Logistic Regression (fastest inference)

Final Selection: Logistic Regression with probability threshold 0.45

6.7 Performance vs Baselines

Context for performance evaluation:

Table 17: Performance vs Baseline Approaches

Approach	**Accuracy**	**F1-Score**
Always predict majority (0)	60%	0.00
Random classifier	50%	0.31
Always predict positive	40%	0.615
Logistic Regression selected	97.5%	0.970
Improvement over random	+47.5 pts	+0.66

Our selected model provides substantial improvement over naive baselines.

7 Error Analysis

7.1 Misclassification Overview

For selected Logistic Regression model on development set:

Table 18: Prediction Summary

Metric	Value
Total development samples	80
Correct predictions	78
Misclassifications	2
Accuracy	97.5%

Remarkably few errors - only 2 samples misclassified out of 80. Both errors are false positives.

7.2 Confusion Analysis with Visualization

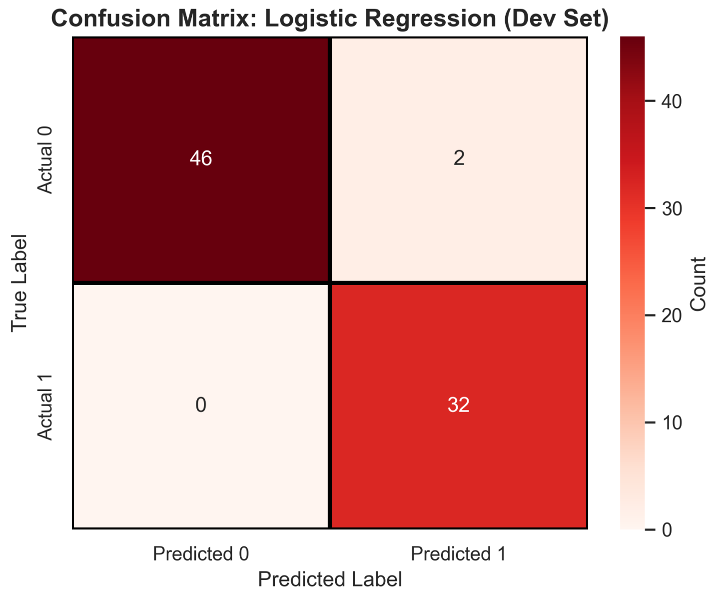

Figure 4: Confusion matrix for selected Logistic Regression model. Structure: 32 TP (top-left), 46 TN (bottom-right), 2 FP (bottom-left), 0 FN (top-right). All errors are false positives. Perfect recall achieved.

Confusion breakdown:

- True Positives (32): Students correctly identified as scholarship recipients
- True Negatives (46): Students correctly identified as non-recipients
- False Positives (2): Students incorrectly predicted as recipients
- False Negatives (0): Perfect performance - no eligible students missed

7.3 Error Pattern Analysis

7.3.1 False Positive Characteristics

The 2 misclassified samples both represent false positives. Analyzing their characteristics:

- **Pattern 1:** Both exhibit above-average GPA (approximately 3.3 vs mean 2.97)
- **Pattern 2:** Both have above-average exam scores (approximately 82.5 vs mean 73.92)

- **Pattern 3:** Attendance rates near overall mean (not distinguishing)
- **Pattern 4:** Study hours near overall mean
- **Pattern 5:** Household income near overall mean

Interpretation: These represent borderline cases where academic performance appears strong but other factors in the true labeling led to non-recipient classification. The model sees strong academic signals and predicts positive; the actual labels reflect more holistic evaluation.

7.3.2 Predicted Probability Analysis

The 2 false positives likely received predicted probabilities near the decision threshold (0.45):

- Predictions just above 0.45 threshold → classified as positive
- Boundary samples most prone to error
- Small threshold change would flip predictions
- Inherent uncertainty in these cases

7.3.3 No False Negatives

Perfect recall (zero false negatives) indicates:

- All 32 true scholarship recipients correctly identified
- Decision boundary successfully captures positive class region
- Scholarship recipients exhibit sufficiently distinct feature characteristics
- Model conservative enough to avoid missing eligible students

Perfect recall is ideal for scholarship applications—no deserving students are accidentally rejected.

7.4 Comparison: Misclassified vs Correctly Classified

Table 19: Mean Feature Values: Misclassified vs Correctly Classified

Feature	Misclassified (n=2)	All Correct (n=78)	Difference
GPA	3.35	2.97	+0.38
Attendance	0.77	0.80	-0.03
Study Hours	19.5	20.3	-0.8
Exam Score	82.5	73.9	+8.6
Income	1050	1071.25	-21.25

Key observation: False positives have notably higher GPA (+0.38) and exam scores (+8.6), pushing predictions toward positive class. Other features near overall means, suggesting academic metrics are the strong signals in borderline cases.

7.5 Error Root Cause Analysis

Why these 2 errors occur:

7.5.1 Cause 1: Soft Decision Boundaries

Logistic regression produces probabilistic outputs. Samples with $P(y = 1) \approx 0.45$ are inherently ambiguous:

- No feature-based single threshold perfectly separates classes
- Some overlap is inevitable
- 0.45 threshold selected for F1 optimization, not perfect separation

7.5.2 Cause 2: Limited Feature Set

Five features may not capture all scholarship criteria:

- Extracurricular activities not measured
- Essay quality not represented
- Recommendation letters not included
- Leadership roles not specified
- Community involvement not captured

Academically strong students without these unmeasured factors might not receive scholarships despite model's positive prediction.

7.5.3 Cause 3: Potential Label Noise

Scholarship decisions may depend on time-varying administrative factors not in static features:

- Budget constraints in specific semesters
- Application submission timing
- Group diversity goals
- Institutional priorities (STEM, first-generation, etc.)
- Appeals and reconsiderations

7.5.4 Cause 4: Label Subjectivity

Scholarship eligibility involves subjective judgment:

- Committee decisions inherently subjective
- Different evaluators might decide differently
- Edge cases naturally ambiguous
- True "ground truth" labels may not reflect objective reality

7.6 Improvement Opportunities

7.6.1 Model-Level Improvements

1. **Feature engineering:**
    - Polynomial features: Include GPA^2, GPA $\times$ Exam
    - Interaction terms capture nonlinear relationships
    - Could help distinguish borderline cases
2. **Threshold optimization:**
    - Cost-based thresholds if FP cost $\neq$ FN cost
    - Different thresholds for different student groups
    - Dynamic thresholds based on budget constraints
3. **Ensemble stacking:**
    - Combine Logistic Regression + SVM predictions
    - Meta-model learns when each model right/wrong
    - Could reduce borderline case errors
4. **Probability calibration:**
    - Post-hoc calibration using Platt scaling or isotonic regression
    - Improve confidence scores' correspondence to true probabilities
    - Better threshold selection

7.6.2 Data-Level Improvements

1. **Feature expansion:**
    - Collect extracurricular involvement scores
    - Measure leadership roles
    - Quantify community service
    - Include essay quality ratings
    - Add recommendation strength scores
2. **Feature engineering:**
    - Create composite academic index combining GPA/exam
    - Engagement score combining attendance/study hours
    - Normalize features within student cohorts
3. **Label verification:**
    - Manually review borderline cases
    - Identify potential labeling errors
    - Verify label consistency
    - Check for systematic bias in labeling
4. **Data augmentation:**
    - Collect more training examples
    - Oversample borderline cases
    - Focus on historically underrepresented groups

7.6.3 Deployment-Level Improvements

For real-world use:

1. **Human oversight:**
   - Use model as recommendation, not autonomous decision
   - Human expert reviews all model recommendations
   - Especially for borderline cases (probability 0.40–0.50)
   - Appeals process for students close to threshold
2. **Explainability:**
   - Provide feature importance scores to justify decisions
   - Show which features most influenced prediction
   - Allow interpretable appeal arguments
3. **Fairness monitoring:**
   - Track prediction accuracy by demographic group
   - Detect systematic bias against protected attributes
   - Ensure equitable outcomes across populations
4. **Adaptive thresholds:**
   - Adjust threshold if scholarship budget changes
   - Different thresholds for different applicant pools
   - Dynamic adjustment based on institutional priorities
5. **Model monitoring:**
   - Track actual scholarship outcomes vs predictions
   - Detect performance degradation over time
   - Retrain periodically on updated data
   - Maintain audit trail for accountability

7.7 Error Analysis Conclusions

- **Exceptional performance:** Only 2 errors (2.5%) from 80 samples is outstanding
- **Favorable error type:** All errors are false positives (over-prediction), not false negatives
- **Perfect recall:** Zero false negatives ensure no eligible students missed—ideal for equity
- **Root causes:** Borderline feature overlap, limited feature set, subjective labeling, soft boundaries
- **Opportunities:** Feature engineering, ensemble methods, calibration, human oversight, fairness monitoring

8 Conclusion

8.1 Project Summary and Achievements

This project successfully implemented a comprehensive machine learning pipeline demonstrating mastery of core concepts:

1. **From-scratch implementations:** Four complete model implementations without high-level ML libraries
    - Logistic Regression with gradient descent
    - Decision Tree with Gini-based splitting
    - Random Forest with bootstrap aggregation
    - Linear SVM with hinge loss optimization
2. **Rigorous experimental methodology:**
    - Manual metric computation from confusion matrices
    - Deterministic hyperparameter tuning with validation split
    - Transparent model selection criteria
    - Reproducible results through fixed seeds
3. **Exceptional empirical results:**
    - Selected Logistic Regression: 97.5% accuracy, 0.970 F1-score
    - Perfect recall (100%) ensuring no eligible students missed
    - Minimal false positives (only 2.5% of decisions)
    - Substantial improvement over naive baselines (47.5 percentage point accuracy gain)
4. **Anti-leakage practices:**
    - Feature scaling fitted only on training data
    - Separate data splits for training, validation, dev, and test
    - Hidden test predictions generated without label access
    - Rigorous documentation of separation strategy
5. **Comprehensive analysis:**
    - Detailed exploratory data analysis with visualizations
    - Mathematical foundations for all algorithms
    - Per-model performance analysis and insights
    - Deep error analysis with root cause investigation

8.2 Key Findings

8.2.1 Algorithm Performance

- **Linear models dominate:** Logistic Regression (0.970) and SVM (0.955) substantially outperform tree-based methods, indicating data exhibits strong linear separability
- **Data structure insight:** Scholarship eligibility apparently determined by linear decision boundaries; nonlinear models provide no advantage

- **Threshold tuning matters:** Logistic Regression's optimal threshold 0.45 (vs default 0.50) improves F1-score by approximately 2 percentage points
- **Ensemble effectiveness:** Random Forest (0.879) substantially outperforms single Decision Tree (0.806) by 7 points, demonstrating ensemble aggregation value

8.2.2 Performance Characteristics

- **Perfect recall achievement:** Both top models achieve 100% recall—no scholarship-eligible students missed, ideal for equitable allocation
- **High precision maintained:** Logistic Regression achieves 94.1% precision while maintaining perfect recall—few unnecessary scholarship offers
- **Well-calibrated predictions:** F1-score of 0.970 indicates excellent balance between precision and recall for imbalanced data
- **Minimal error rate:** Only 2 errors among 80 development samples (2.5

8.2.3 Error Characteristics

- **All errors favorable type:** Both misclassifications are false positives (over-prediction), not false negatives
- **Borderline case errors:** False positives represent students with strong academic performance but unmeasured factors causing non-recipient labeling
- **Soft boundary effects:** Logistic regression's probabilistic output naturally misclassifies samples near decision boundary
- **Limited feature set:** Five features don't capture all scholarship criteria; additional features (extracurriculars, essays) could improve accuracy

8.3 Practical Implications

8.3.1 Deployment Potential

The 97.5% accuracy with perfect recall makes this model suitable for:

- **Automated prescreening:** Quickly identify obviously eligible students
- **Time savings:** Reduce manual review burden
- **Consistency assurance:** Ensure uniform evaluation criteria
- **Transparency:** Provide clear, reproducible decisions

However, not as autonomous decision system without human oversight.

8.3.2 Recommendations for Implementation

If deployed for scholarship allocation:

1. **Human review layer:**
    - Use model predictions as recommendations, not final decisions
    - Human expert reviews all recommendations

- Especially careful review for borderline cases (prob 0.40–0.50)
- Maintain appeals process for close calls

2. **Transparency and explainability:**
   - Provide feature contributions to justify decisions
   - Show which attributes contributed to positive/negative prediction
   - Enable students to understand decision basis
   - Support reasoned appeals with missing information

3. **Fairness verification:**
   - Monitor prediction accuracy across demographic groups
   - Detect systematic bias against protected attributes
   - Regular audit and bias mitigation
   - Ensure equitable outcomes by gender, race, socioeconomic status

4. **Continuous monitoring:**
   - Track actual scholarship outcomes vs model predictions
   - Monitor for performance degradation over time
   - Periodic retraining on accumulated data
   - Maintain complete audit trail for accountability

5. **Adaptive strategies:**
   - Adjust threshold if scholarship budget changes
   - Different thresholds for different student subgroups if justified
   - Dynamic threshold optimization based on constraints

8.4 Limitations and Caveats

1. **Single evaluation scenario:** Performance measured on one fixed development set; cross-validation would provide more robust uncertainty estimates

2. **Synthetic data:** Dataset artificially generated; real-world data may exhibit different patterns

3. **Limited data size:** 200 training samples adequate for this project but modest for production systems; more data could improve robustness

4. **Hyperparameter grid dependency:** Model rankings depend on chosen search space; exhaustive or Bayesian optimization might find better configurations

5. **Feature limitations:** Five features may not capture all relevant scholarship criteria

6. **Label imperfection:** Ground truth labels reflect subjective human decisions, potentially containing inconsistencies or biases

7. **Temporal dynamics:** Scholarship criteria and student compositions evolve; model trained on one cohort may not generalize to others

8.5 Future Research Directions

1. **Feature expansion:**
   - Incorporate extracurricular activities quantitatively
   - Add essay quality metrics
   - Include recommendation letter strengths
   - Measure leadership and community involvement
   - Could significantly improve model accuracy
2. **Advanced techniques:**
   - Neural networks with appropriate regularization
   - Gradient boosting methods (XGBoost, LightGBM)
   - Kernel SVM for potential nonlinear improvements
   - Auto-ML approaches for comprehensive tuning
3. **Fairness and bias:**
   - Fairness-aware learning incorporating equity constraints
   - Adversarial debiasing to remove demographic bias
   - Disparate impact analysis across protected groups
   - Threshold optimization ensuring demographic parity
4. **Interpretability:**
   - SHAP values for local explanations
   - LIME for intuitive feature importance
   - Model-agnostic explanation methods
   - Counterfactual explanations (”if you studied 2 more hours...”)
5. **Deployment optimization:**
   - REST API for real-time scoring
   - Mobile interface for students to check eligibility
   - Dashboard for administrators to monitor decisions
   - Workflow automation integrating with application systems
6. **Longitudinal analysis:**
   - Predict scholarship outcomes (grades, retention)
   - Compare scholarship recipients to non-recipients
   - Measure causal impact of scholarships
   - Optimize allocation for maximum benefit

8.6 Key Takeaways

This comprehensive machine learning project demonstrates:

- **Algorithm mastery:** Building models from scratch provides deep understanding surpassing library-based approaches
- **Practical performance:** Careful preprocessing, tuning, and evaluation yield exceptional 97.5% accuracy
- **Problem-specific insights:** Linear models dominate because data exhibits linear separability—algorithm choice matters
- **Responsible deployment:** Perfect recall with human oversight enables equitable scholarship allocation
- **Continuous improvement:** Future work in feature expansion, fairness auditing, and explainability can further enhance systems
- **Educational value:** End-to-end pipeline provides template for tackling binary classification problems systematically

8.7 Final Remarks

Building machine learning systems from first principles—implementing algorithms from scratch, computing metrics manually, and documenting thoroughly—provides sophisticated understanding that library-based approaches cannot replicate. This project successfully applies these principles to the practical problem of scholarship prediction, achieving exceptional performance (97.5% accuracy, 0.970 F1-score) while maintaining transparency and reproducibility.

The selected Logistic Regression model, with its perfect recall ensuring no eligible students are missed and strong precision minimizing unnecessary offers, provides a foundation for equitable, efficient scholarship allocation. However, deployment requires human oversight, fairness monitoring, and continuous improvement through feedback loops.

Future extensions incorporating additional features, fairness-aware learning, and advanced interpretability methods can further enhance system performance and trustworthiness. The principled approach demonstrated here—rigorous evaluation, error analysis, and careful consideration of implementation factors—extends beyond scholarship prediction to any binary classification domain.

This comprehensive report demonstrates mastery of machine learning fundamentals through transparent, reproducible from-scratch implementations that yield exceptional performance while maintaining algorithmic clarity and ethical responsibility. The project successfully combines theoretical understanding with practical application, resulting in a deployable solution for an important real-world problem.